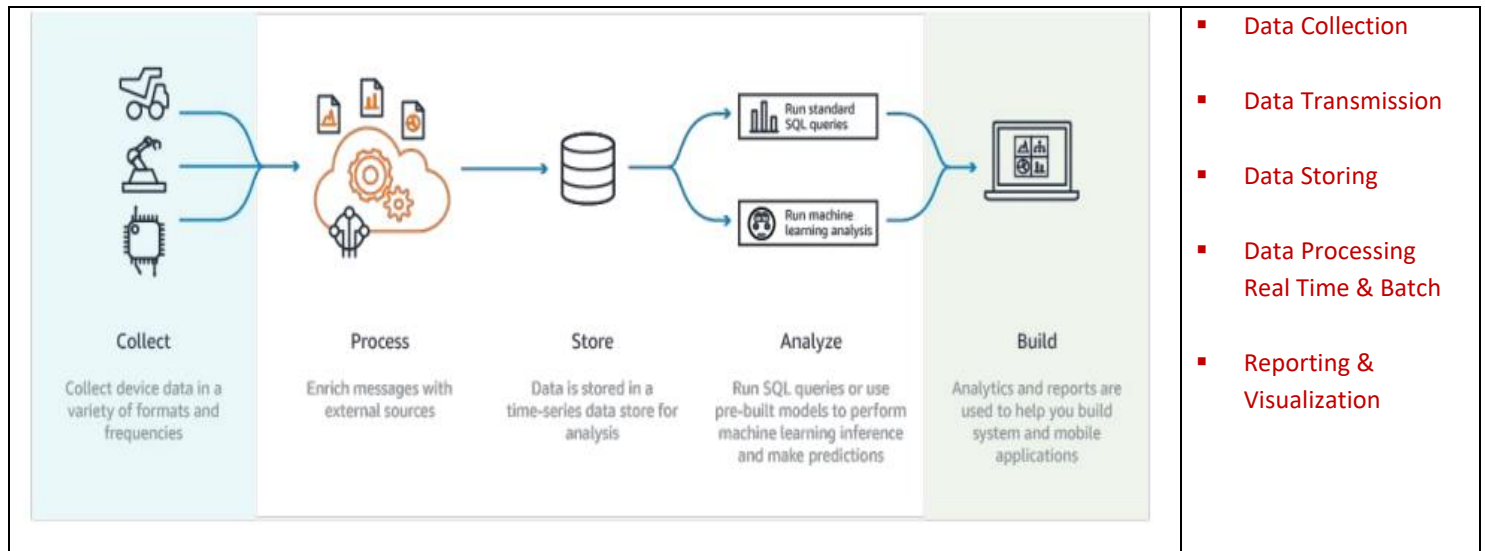


IoT Cloud Processing & Analytics - AWS IoT Core and Dynamo DB

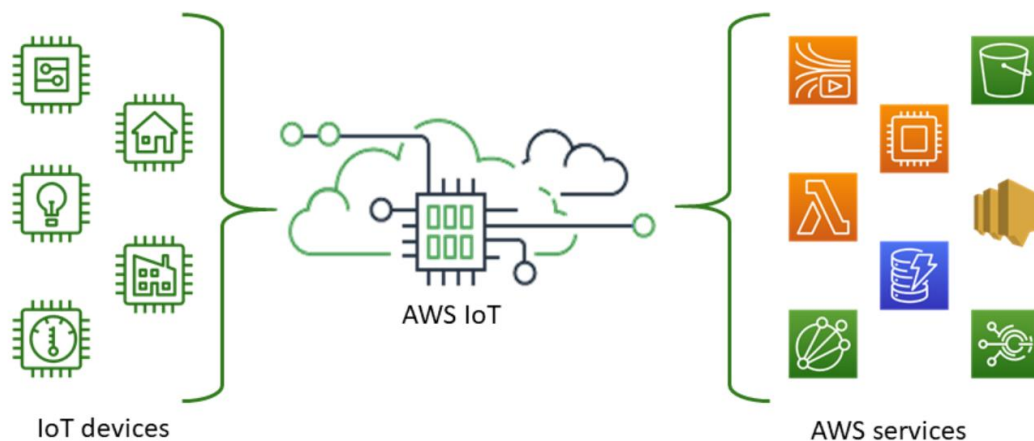
IoT Data Lifecycle



AWS IoT Services

Device & Edge Services	Cloud Service for IoT	Analytics Service
Greengrass :- SDK for Edge Servers FreeRTOS - Real-time OS for Devices FreeRTOS BLE Mobile SDK - Low power communication via mobiles	• IoT Core - Scalable and secure device connection and communication, Device shadows, On-the-fly data transformation, post-collection seamless connection to various AWS services • IoT Device Management - Registration, Organization, Monitoring, Update Management	• AWS IoT Analytics - Workflows, Time-series data storage, Queries, Transformations, ML • IoT Events - Rule-based event processing and actions • IoT Things Graph - Visual organization and workflows

AWS IoT Core



<https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html>

<https://aws.amazon.com/iot-core/>

AWS-IOT

- Managed IoT services by AWS
- Connect and manage billions of devices.
- Bridge between Devices & AWS Services
- Collect, store, & analyse IoT data

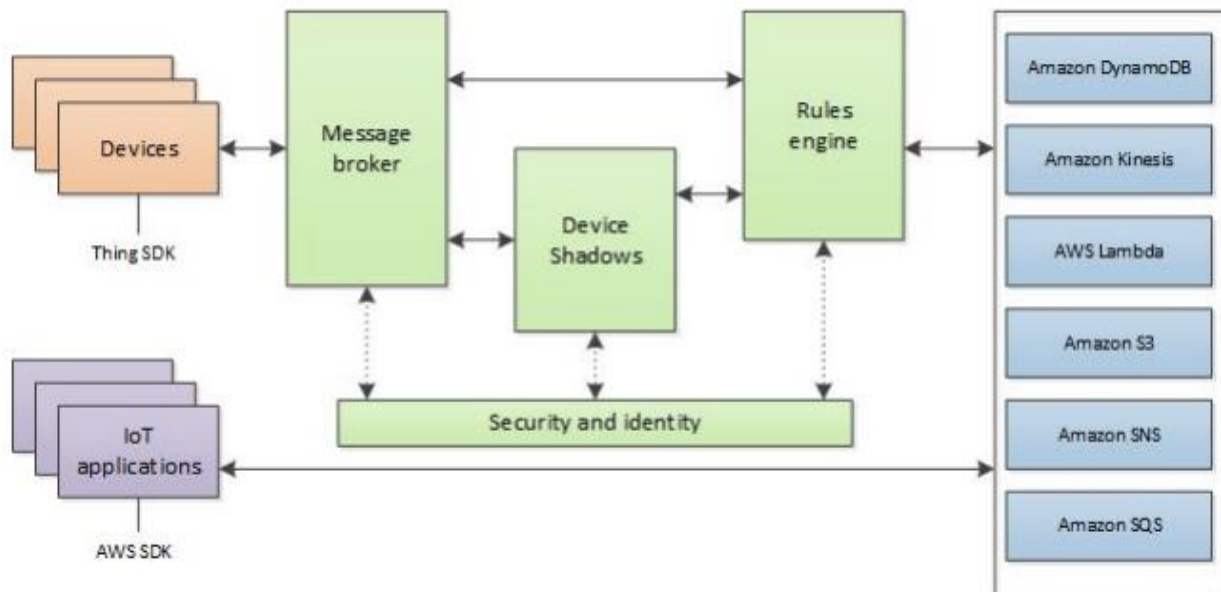
Similar Service with Azure & GCP

- Azure IoT Hub
- Google Cloud IoT Core

IOT Core

- Virtual device configuration and mapping to real devices
- Shadow device service to maintain status, monitoring, updates, and upstream connections
- Device organization in various hierarchical categories
- Secure device communication - X.509 certificate and keys for authentication and authorization
- Communication protocols - MQTTS, MQTT over secure websockets, HTTPS, LoRaWAN
- On-the-fly data transformation
- Data streaming to various services - DynamoDB, S3, SNS, Lambda, Elasticsearch, Cloudwatch, IoT Analytics, IoT Events, Kinesis, SQS, Custom services
- Scalable to billions of devices

AWS IOT Core Components



Thing	Is the the basic building block. Thing. A Thing can be an end device generating sensor data using one or more sensors. Edge devices using Greengrass SDK can be configured as a Greengrass core device.
IoT Device Gateway & Message broker	- Multiple Communication protocols - MQTTS, MQTT over secure websockets, HTTPS, LoRaWAN - Low latency scaling to billions of devices - Bidi communication with devices - data ingress, heartbeat monitoring, control messages, updates
IoT Device Registry	- Virtual device registration and Bulk provisioning with hierarchies (using API) - Secure communication automated API-driven provisioning & management
IoT Device Shadow	- Additionally, you can also set the initial parameters that should be part of the device shadow. - State management with continuous updates - Bidi communication for downstream configuration and software updates - Status available to upstream services even on device disconnections - Device state restoration on reconnection and supports generic Get, Update , Delete REST api calls https://docs.aws.amazon.com/iot/latest/developerguide/iot-device-shadows.html
Jobs Management and API	- Defined jobs for devices - OS updates, software installs, reboot, cache reset, troubleshooting information, security certificate updates - Extensive control over device targeting and job ordering - Support for supplying assets including new updated files and configuration information
IoT Rule Engine	- Enables automatic upstream data and event communication to various services - DynamoDB, S3, SNS, Lambda, Elasticsearch, Cloudwatch, IoT Analytics, IoT Events, Kinesis, SQS, Custom services - SQL based selection syntax for - Partial payload pickup possible - select value, device_id, timestamp - Topic selection control - from devices/# . Possible data pickup or alerts, based on rules - where value >= 50 - Use cases - Storage, Stream Processing, Direct alerts/notifications, Processing triggers based on conditions, etc.
DynamoDB action rule	Automatically send all (or some) device data to DynamoDB . Map different set of topics to different tables for automatic segregation. Complete payload in one column or map payload keys to different columns

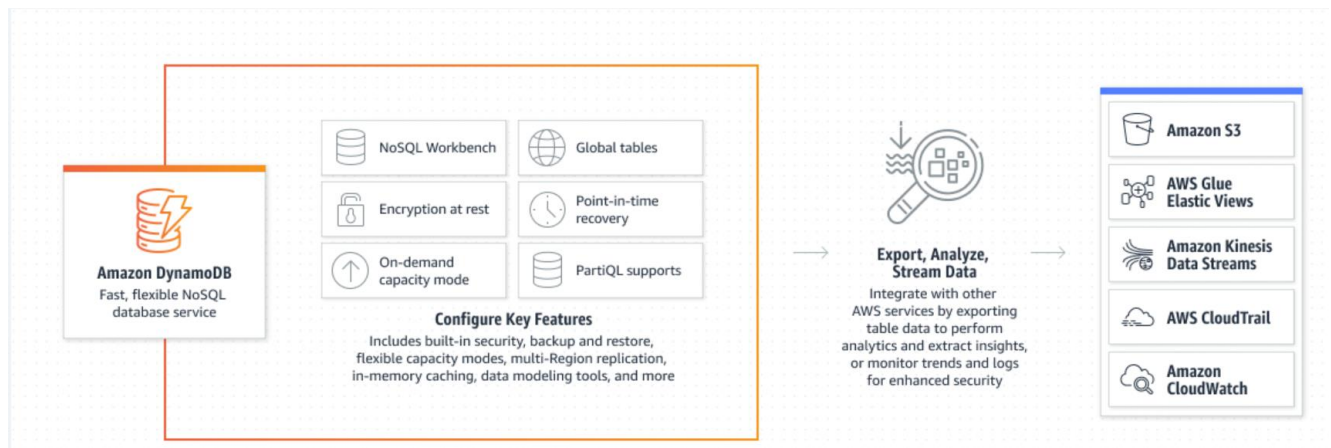


AWS Dynamo DB

NoSQL DB :- <https://www.mongodb.com/resources/basics/databases/nosql-explained>

AWS Dynamo DB - <https://aws.amazon.com/dynamodb/>

Dynamo VS Document DB - <https://dynobase.dev/dynamodb-vs-documentdb/>



❖ **Fully Managed Serverless NoSQL DB by AWS** – No installation, upgrade, capacity planning, or manual provisioning needed

- Auto Scalable with simple rules based on size and read/write load
- Redundant - No single point of failure, multi-zone & multi-region
- High Performance with nearly unlimited throughput and storage
- Very low latency – as it has hash maps – fast retrieval of data
- Secure transmission, encrypted at rest, Backup/Restore

❖ **NoSQL data model:-**

- Collection (tables in RDBMS)
- items (row in RDBMS) ;
- items has attributes (equivalent to column); supports nested attribute up to 32 levels
- Access based optimization - Partition and Sort keys for efficient single or range pickup

❖ **Ingress / Egress:-**

- Ingress - Simple ingestion setup from IoT Core and other services
- Egress - Base for batch processing and analytics services, item-level change streaming to Kinesis

References

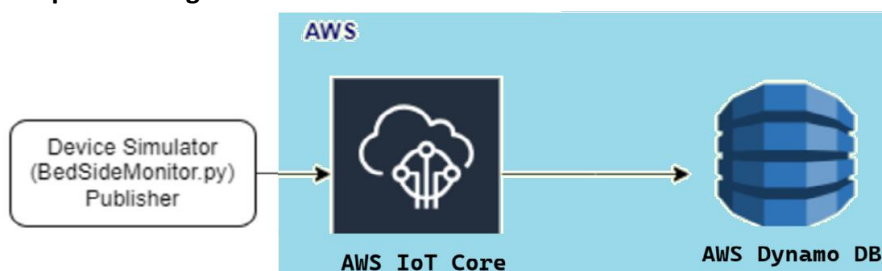
<https://docs.aws.amazon.com/iot/latest/developerguide/what-is-aws-iot.html>
<https://aws.amazon.com/iot-core/>
<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>

Hands On

Practice Material Provided

1. Step by Step guide - 4.1-Main-IoT-Core-BedSideMonitor-Example
2. BedSideMonitor.py – Python Source code by Device Simulator

Component Diagram



AWS IoT Core Components	BedsideMonitor.py (python code for device simulator)	Dynamo DB
- Thing - Name, Group, Type - Policy - Device Shadow - MQTT Test Client - Dynamo DB - Role - Thing Rule to insert incoming data in MQTT into DB	- For connecting to AWS Core - End point of Thing - Security certificates - Topic - Python library - AWSIoTPythonSDK - Boto3	- Role - Collection